INFORMATION MANAGEMENT SCHOOL

Project Guidelines

First period

NOVA IMS

Bachelor's Degree in Data Science

Capstone Project

2026-2027

Miguel Brinkmann Cardoso

Contents

The project **2**

Dates and weight **2**

What the project must contain **2**

Required components . . . 2
Recommended . . . 3
Compliance . . . 3
Balance . . . 3

Technical choices **4**

How the project is evaluated **4**

What is assessed . . . 4
Principles, stated in advance . . . 5

Deliverables **5**

1. Project proposal, Sunday 4 October . . . 5
2. Technical proposal, Sunday 6 December . . . 5
3. Final submission, Sunday 3 January . . . 6
4. Defence . . . 6

How to build it **6**

Resources **6**

The project

Your team of up to four builds a working AI application that solves a real problem, deploys it at a public address, and defends it. The project is where everything taught in the course comes together.

It shows that you can:

- design and build an application in which an AI model does real work
- work as a team on one codebase
- structure that codebase in layers, so each part has one job
- deploy it, and see what it does in production through tracing
- explain and defend every decision in it, individually

Dates and weight

Milestone	When	Weight
Project proposal	Sunday 4 October, 23:59	Assessed with the technical proposal
Technical proposal	Sunday 6 December, 23:59	15%, for both proposals
Final submission	Sunday 3 January, 23:59	75%, together with the defence
Defence	To be announced	

The remaining 10% of the course grade comes from the in-class quizzes. The second call is graded 100% on delivery and defence, under separate guidelines.

The Week 3 proposal and the role plans in it are drafts. Both can change until the technical proposal.

What the project must contain

Required components

Every project has each of these components. Check your project against each one.

1. AI integration. A model does real work in your application, steered by your prompts and system instructions. An answer the user reads can be text. An answer your code acts on, such as fields extracted from a document or a choice of what happens next, comes back as structured output against a schema, not as text you parse.

2. An agent the user talks to. A multi-turn conversation that drives the application. The agent decides which of your tools to call, chains several calls when one request needs them, and uses what was said earlier in the conversation. A workflow, in which your code decides each step, can be part of the application but does not meet this requirement.

3. Tools. Functions of yours that the agent calls to read and change the application's data, each documented in `docs/TOOLS.md`. A tool that fails returns a result the agent can act on instead of crashing the conversation. Calls that fail transiently and are safe to repeat are retried. A tool that deletes or overwrites asks the user first, unless the user can undo it.

4. Documents. The application reads documents, such as text files, PDFs or images, and uses their content for something the user cares about: as the core feature, or to ground answers in a knowledge base.

5. Layered architecture. Interface, application logic, AI access and tools live in separate layers, each with one job, so that any of them can be changed or tested without touching the others. Prompts are built and decisions made in the logic, never in the interface. Configuration, prompts and schemas each live in their own module: changing the model means editing one line.

6. Tracing. Every user request produces one trace, with a span for each model call and each tool call. Traces show the model, timing, tokens and errors, mark the failures the code handles, and say how each request ended and which conversation it belongs to. The team decides, and states, whether prompts and responses are recorded and how long traces are kept. It can open the trace of a failure it found and show what it changed because of it.

Recommended

- **Images and PDFs as input**, when the problem involves them.
- **Semantic search**, where the problem needs it.
- **An evaluation set.** Ten to twenty inputs for your core AI task, each with the output you expect, scored by a script you re-run whenever you change a prompt or a model.
- **Beyond this course.** Bringing in what you learnt in other courses, such as a machine-learning model, text mining or data analysis, is encouraged and valued where it serves the problem.

Compliance

The application tells its users that they are interacting with an AI system, as Article 50 of the EU AI Act requires, and handles personal data as the GDPR requires. The terms of the services you use apply too.

Balance

Scope the project to what your team can finish, deploy and defend. Add a technique where it serves the problem, not to tick a box. Equally, do not avoid a technique the course taught because it takes effort, and do not complicate something that already works. A well-executed application that applies the right techniques scores higher than one that forces every technique in.

Technical choices

What is taught in class is a recommendation. What the project requires is that each piece exists and that you can justify your choice.

Piece	Required	In class
Backend	Python	Python
LLM	any major provider, through the provider's current SDK or a framework agreed beforehand	Google Gemini (`google-genai`) or Anthropic Claude (`anthropic`)
Interface	any	Streamlit
Tracing	any; one trace per request, from the deployed application too	Langfuse Cloud EU, through its Python SDK; Arize Phoenix, on plain OpenTelemetry, as a local alternative
Packaging	every dependency pinned to an exact version	uv, with its lock file
Deployment	a public address, working at the defence	Streamlit Community Cloud

Gemini's older SDK, `google-generativeai`, has been unsupported since November 2025. A project built on it does not meet this requirement.

Which key backs the deployed application is your choice: a free tier, your own credits, or the Anthropic keys provided for the course.

Agree any choice outside what is taught with the instructor, in class or at mcardoso@novaims.unl.pt, before you build on it.

How the project is evaluated

What is assessed

- **Technical implementation.** The required components, error handling, and whether you understand the APIs you use.
- **Functionality.** The application works and solves the problem it states; its features make sense for that problem, and it is clear to use.
- **Innovation and effort.** Going beyond the minimum where it adds value; creative solutions; research beyond class material.
- **Documentation and code.** README, architecture and tool documentation that match the code; setup instructions that work; clean, organised code.
- **Work in git.** How well the team organises its work in the repository.
- **Technical proposal against delivery.** What the technical proposal committed to, compared with what was delivered.
- **Defence.** A pitch that makes the case for the product, a live demonstration of its key features, and answers that defend the technical decisions.

- **Individual understanding.** Each member can explain the product, the course content it relies on and any part of the code, justify the decisions behind it, and discuss the trade-offs and the alternatives considered.

Principles, stated in advance

A working simple solution beats a sophisticated broken one. A broken core feature affects the whole grade, not one line of it.

Every member answers for the whole project. At the defence, any member may be asked about the product, the course content it relies on, or any part of the code, not only the parts they built. A member who cannot answer loses marks, and the team's grade may suffer too, however well the application works.

Individual grades can differ from the team grade, according to each member's defence.

You may use AI tools to write code. That code is held to the same rule as code you typed: every member must be able to explain it.

Credit what you build on. Code taken from another project is credited in the README, as open-source projects do, and the team is assessed on what it did with it. Documentation, examples and cookbooks need no credit.

Late submission. The deadline is at 23:59. The final submission is accepted for three days after it, with points deducted from the grade out of 20; each of those days also ends at 23:59.

Submitted	Deduction
The first day after the deadline	-2 points
The second day	-5 points
The third day	-9 points
Later	Not accepted

What counts is the repository's default branch and the Moodle submission. Work committed after the deadline is treated as late, and work left on another branch is not part of the submission.

Deliverables

1. Project proposal, Sunday 4 October

See PROPOSAL_WEEK_03.md.

2. Technical proposal, Sunday 6 December

The problem and your solution, each member's role and responsibilities, a technical blueprint with an architecture diagram, and a realistic scope and timeline. See TECHNICAL_PROPOSAL_GUIDELINES.md.

3. Final submission, Sunday 3 January

- **Repository.** GitHub, with `mb-cardoso` added as a collaborator. It contains:
  - the complete source code
  - `README.md`, following README_TEMPLATE.md or an equivalent structure, with setup instructions that work on a fresh clone and a note on how the application is deployed
  - `docs/ARCHITECTURE.md`: the architecture and the reasons behind it
  - `docs/TOOLS.md`: each tool's purpose, parameters, return value and failures
  - `.env.example`, listing every environment variable; no key or secret is ever committed
  - any further documentation someone would need to understand and maintain the project
- **Moodle.** The same code as a zip, named `TeamName_FinalProject.zip`.
- **Deployment.** The public address in the README, working at the defence.

4. Defence

An investor pitch and live demonstration: present the application as a startup seeking investment, in the format of your choice. Then technical questions. The date, how long each part lasts and each team's slot are announced on Moodle. Every member attends and answers questions about the product, the course content and any part of the code, not only their own.

How to build it

- Get one interaction working end to end, from the user's message to the answer, before you add the next.
- Read every line an AI tool writes before you keep it. It can call an SDK that no longer exists, or do something other than what you asked.
- Trace from the first working interaction, not at the end.
- Deploy early, and redeploy as you go. A first deployment finds the missing dependencies and paths that a defence would otherwise find.
- Before you submit, clone the repository into an empty folder and follow your own README. If it does not run that way, it will not run when it is graded.

Resources

Documentation

- Google AI Studio, for API keys: https://aistudio.google.com/
- Gemini API: https://ai.google.dev/gemini-api/docs
- Google GenAI Python SDK: https://googleapis.github.io/python-genai/
- Streamlit: https://docs.streamlit.io/
- Streamlit Community Cloud: https://docs.streamlit.io/deploy/streamlit-community-cloud

- OpenTelemetry for Python: https://opentelemetry.io/docs/languages/python/
- OpenTelemetry GenAI conventions (in Development): https://opentelemetry.io/docs/specs/semconv/gen-ai/
- Langfuse Python SDK: https://langfuse.com/docs/observability/sdk/overview
- Arize Phoenix: https://arize.com/docs/phoenix
- uv: https://docs.astral.sh/uv/

Other LLM providers

- OpenAI: https://developers.openai.com/api/docs
- Anthropic: https://platform.claude.com/docs

Data, if your project needs it

- Files (JSON, CSV)
- SQLite, built into Python and file-based: https://docs.python.org/3/library/sqlite3.html
- PostgreSQL: https://www.postgresql.org/, with psycopg: https://www.psycopg.org/
- MongoDB Atlas: https://www.mongodb.com/docs/atlas/, and its vector search: https://www.mongodb.com/docs/vector-search/